

Projet de Fin d'Études

LeadGen Francophone 360+

Plateforme automatisée de génération
de leads B2B IT pour Solvinya Group

Spécialité : Génie Logiciel — Intelligence Artificielle

Préparé par : Ferjani Malek
Encadrant académique : Encadrant ESPRIT
Encadrant entreprise : Équipe Solvinya Group
Année universitaire : 2024 – 2025

Python 3.11 | FastAPI | PostgreSQL | n8n | Docker | Groq LLM

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous :

Nom & Prénom : Ferjani Malek

Encadrant Entreprise / Business Site Supervisor

• Nom & Prénom :

Cachet & Signature :

Encadrant Académique / Academic Supervisor

• Nom & Prénom :

Signature :

Ce formulaire doit être rempli, signé, scanné et inséré après la page de garde.

Dédicaces

*À mes parents, pour leur soutien indéfectible
et leur confiance tout au long de ce parcours.*

*À tous ceux qui croient que la technologie
peut simplifier le travail humain
et créer de la valeur réelle.*

— Ferjani Malek

Remerciements

Ce projet de fin d'études représente l'aboutissement d'un travail intensif alliant développement logiciel, traitement automatique du langage naturel, et automatisation des processus. Sa réalisation n'aurait pas été possible sans le soutien de plusieurs personnes à qui je tiens à exprimer ma sincère gratitude.

Je remercie tout d'abord mon **encadrant académique** pour ses conseils méthodologiques, ses retours critiques et sa disponibilité tout au long de ce projet.

Je remercie également l'équipe de **Solvinya Group** pour m'avoir confié ce projet ambitieux et m'avoir fourni le contexte métier nécessaire à sa bonne réalisation. La liberté technique accordée m'a permis d'explorer des solutions innovantes combinant scraping, NLP et LLM.

Mes remerciements vont aussi à la communauté open source — les projets *FastAPI*, *linkedin-api*, *scikit-learn*, *n8n* et *PostgreSQL* — dont les outils constituent la colonne vertébrale de cette plateforme.

Enfin, je remercie ma famille et mes proches pour leur encouragement constant durant ces mois de développement.

Table des matières

Remerciements	ii
Introduction générale	1
1 Cadre du projet	3
1.1 Introduction	4
1.2 Présentation de l'organisme d'accueil	4
1.2.1 Solvinya Group	4
1.2.2 Domaines d'activités	4
1.3 Présentation du projet	5
1.3.1 Problématique	5
1.3.2 Étude de l'existant	6
1.3.3 Solution proposée	6
1.4 Méthodologie de travail	6
1.4.1 Choix de la méthodologie Scrum	6
1.4.2 Planification des sprints	7
1.5 Conclusion	7
2 Analyse des besoins et architecture globale	8
2.1 Introduction	9
2.2 Analyse des besoins	9
2.2.1 Besoins fonctionnels	9
2.2.2 Besoins non fonctionnels	10
2.3 Acteurs du système et cas d'utilisation	10
2.3.1 Identification des acteurs	10
2.3.2 Diagramme des cas d'utilisation	11
2.4 Architecture globale du système	11
2.4.1 Vue macro	11
2.4.2 Technologies et choix techniques	12
2.5 Les 5 types de signaux B2B	13
2.6 Conclusion	13

3	Conception technique	14
3.1	Introduction	15
3.2	Architecture des collecteurs	15
3.2.1	Collecteur LinkedIn Jobs — Signal d’externalisation	15
3.2.2	Collecteur LinkedIn B2B — Signaux directs	15
3.2.3	Collecteur BOAMP	16
3.2.4	Collecteur TED EU	17
3.3	Pipeline d’ingestion	17
3.3.1	Vue d’ensemble — 6 étapes, ≈ 50 ms/prospect	18
3.3.2	Formule de scoring — 4 dimensions pondérées	18
3.4	Module NLP	19
3.4.1	Architecture des deux classifieurs	19
3.4.2	Comparaison des modèles	20
3.4.3	Distribution des données d’entraînement	20
3.5	Générateur de messages LLM	21
3.6	Schéma de base de données	22
3.7	Conception des workflows n8n	22
3.7.1	Workflow 04 — Déclencheur collecte (principal)	22
3.7.2	Workflow 03 — Démo live (pipeline bout-en-bout)	23
3.8	Conclusion	23
4	Réalisation et résultats	24
4.1	Introduction	25
4.2	Environnement de développement	25
4.2.1	Stack technique	25
4.2.2	Architecture Docker Compose	25
4.3	Implémentation des collecteurs	26
4.3.1	Gestionnaire de sessions LinkedIn	26
4.3.2	Déduplication dans le pipeline	26
4.3.3	Collecteur TED EU — gestion du multilinguisme	27
4.4	API REST FastAPI	28
4.4.1	Endpoint de collecte asynchrone	28
4.4.2	Endpoint NLP	28
4.5	Interfaces utilisateur	29
4.5.1	Dashboard Metabase	29
4.5.2	Interface Swagger (FastAPI /docs)	29
4.6	Résultats et évaluation	29
4.6.1	Volume de données collectées	29
4.6.2	Répartition par niveau de priorité	31

4.6.3	Scores moyens par pays (LinkedIn Jobs)	31
4.6.4	Performances du module NLP	31
4.6.5	Performances du pipeline end-to-end	32
4.6.6	Exemple de prospect généré	32
4.7	Tests	33
4.8	Conclusion	33
	Conclusion générale	34
	Bibliographie	36

Table des figures

1.1	Les 5 formes de signaux d'un besoin IT actif	5
2.1	Diagramme des cas d'utilisation	11
2.2	Architecture macro du système LeadGen 360+	11
3.1	Flux interne du collecteur LinkedIn Jobs	15
3.2	Stratégie et flux du collecteur LinkedIn B2B	16
3.3	Les 6 étapes du pipeline <code>ingest.py</code>	18
3.4	Décomposition du score pour 4 exemples types	18
3.5	Architecture des deux pipelines NLP (TF-IDF + LinearSVC)	19
3.6	Comparaison des approches NLP — corpus de 604 exemples IT	20
3.7	Répartition des 604 exemples d'entraînement NLP par secteur	20
3.8	Pipeline de génération de messages via LLM	21
3.9	Schéma entité-relation de la base de données	22
3.10	Workflow 04 — Déclencheur de collecte automatique	22
3.11	Workflow 03 — Démo live bout-en-bout (8 nœuds)	23
4.1	Architecture Docker Compose — 5 services	26
4.2	Volume de prospects collectés par source et par pays (2 mai 2026)	30
4.3	Distribution des prospects par plage de score	31
4.4	Score moyen par pays — collecteur LinkedIn Jobs	31
4.5	Latences du pipeline de traitement bout-en-bout	32

Liste des tableaux

1.1	Profil de Solvinya Group	4
1.2	Comparaison des solutions de génération de leads B2B	6
1.3	Planification Scrum du projet	7
2.1	Besoins fonctionnels du système	9
2.2	Besoins non fonctionnels	10
2.3	Stack technique et justifications	12
2.4	Les 5 types de signaux détectés	13
3.1	Patterns regex de détection B2B et leurs scores de boost	16
3.2	Table des scores par dimension	19
4.1	Environnement technique complet	25
4.2	Résultats détaillés par source (au 2 mai 2026)	30
4.3	Performances des modèles NLP	31
4.4	Tableau de synthèse des tests	33

Introduction générale

Contexte

Dans un marché des services numériques en forte croissance, les entreprises de type ESN (Entreprise de Services Numériques) font face à un défi commercial récurrent : identifier rapidement et avec précision les organisations qui ont un besoin **actif et budgété** en prestations IT. Ce besoin peut se manifester sous différentes formes : un appel d'offres public, une demande de proposition (RFP), une recherche de partenaire technologique, ou encore — de façon indirecte — le recrutement d'un profil IT rare que l'entreprise peine à trouver en CDI.

L'identification manuelle de ces signaux est chronophage, non systématique, et fortement dépendante de l'intuition et du réseau personnel des commerciaux. Plusieurs heures de veille hebdomadaire sur LinkedIn, le BOAMP, et les journaux officiels ne produisent que quelques leads de qualité incertaine.

Objectifs du projet

Ce rapport présente **LeadGen Francophone 360+**, une plateforme de génération automatisée de leads B2B IT, développée pour Solvinya Group et ciblant les marchés francophone : France, Belgique, Luxembourg, Suisse, et Tunisie.

Les objectifs principaux sont :

1. **Collecter** automatiquement les signaux B2B depuis 5 sources (LinkedIn Jobs, LinkedIn B2B Posts, BOAMP, TED EU, Malt.fr) ;
2. **Qualifier** chaque prospect avec un score 0–100 basé sur des règles métier déterministes (pays, technologie, secteur) ;
3. **Classifier** le texte des opportunités par secteur et niveau de priorité via un modèle NLP (TF-IDF + LinearSVC) ;
4. **Générer** des messages de prospection personnalisés prêts à envoyer via un LLM (Groq, modèle llama-3.1-8b-instant) ;
5. **Automatiser** l'ensemble du pipeline avec n8n et exposer une API REST FastAPI.

Organisation du rapport

Ce rapport est structuré en quatre chapitres :

Chapitre 1 expose le cadre du projet : présentation de Solvinya Group, problématique détaillée, solution proposée et méthodologie de développement adoptée.

Chapitre 2 présente l'analyse des besoins fonctionnels et non fonctionnels, les cas d'utilisation, et l'architecture globale du système.

Chapitre 3 décrit la conception technique détaillée : architecture des collecteurs, pipeline d'ingestion, formule de scoring, module NLP, générateur LLM, schéma de base de données et workflows n8n.

Chapitre 4 présente la réalisation : environnement technique, implémentation de chaque composant, tests et résultats obtenus (815 prospects collectés au 2 mai 2026).

Chapitre 1

Cadre du projet

1.1 Introduction

Ce premier chapitre établit le cadre général du projet. Nous présentons l'entreprise d'accueil Solvinya Group, détaillons la problématique commerciale qui justifie ce développement, exposons la solution proposée, et décrivons la méthodologie de développement adoptée.

1.2 Présentation de l'organisme d'accueil

1.2.1 Solvinya Group

Solvinya Group est une **ESN** (Entreprise de Services Numériques) opérant principalement sur les marchés francophones : France, Belgique, Luxembourg, Suisse et Tunisie. L'entreprise propose des services de conseil, de développement logiciel, d'externalisation IT et d'accompagnement en transformation numérique.

Critère	Description
Secteur	ESN — Entreprise de Services Numériques
Marchés cibles	France, Belgique, Luxembourg, Suisse, Tunisie
Services	Développement logiciel, Conseil IT, Externalisation
Technologies	COBOL/Mainframe, SAP, Java, Python, DevOps, Cloud
Clients types	Banques, Assurances, Groupes industriels, Secteur public

Table 1.1. Profil de Solvinya Group

1.2.2 Domaines d'activités

Solvinya Group intervient principalement dans les domaines suivants :

- **Systèmes legacy** : maintenance et migration de systèmes COBOL/Mainframe dans le secteur bancaire ;
- **Intégration ERP** : déploiement et personnalisation de SAP ;
- **Développement applicatif** : Java, Python, React ;
- **Data & IA** : projets Data Engineering, Machine Learning ;
- **DevOps & Cloud** : CI/CD, infrastructure AWS/Azure.

1.3 Présentation du projet

1.3.1 Problématique

La croissance de Solvinya Group repose sur sa capacité à identifier et contacter rapidement les entreprises qui ont un **besoin IT actif**. Ce besoin peut prendre plusieurs formes (illustrées en figure 1.1) :

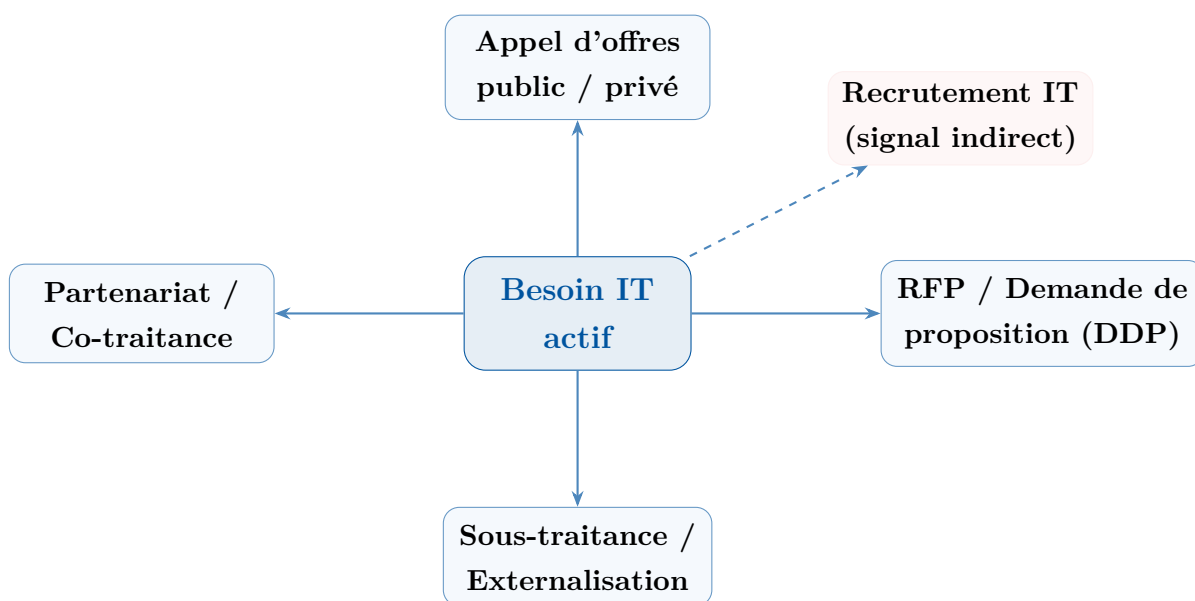


Figure 1.1. Les 5 formes de signaux d'un besoin IT actif

État actuel (sans l'outil) : les commerciaux effectuent manuellement cette veille, ce qui représente :

- 3 à 5 heures par semaine de recherche manuelle sur LinkedIn, BOAMP et journaux officiels ;
- Un processus non systématique, dépendant du réseau personnel ;
- Des leads de qualité variable, souvent périmés au moment de la prise de contact ;
- Absence totale de couverture des marchés belge, luxembourgeois et suisse.

1.3.2 Étude de l'existant

Solution	Points forts	Limites	Coût
LinkedIn Sales Navigator	Interface intuitive, ciblage	Manuel, 99/mois, pas d'AO publics	Élevé
Hunter.io	Enrichissement email	Pas de détection de signaux	Moyen
Apollo.io	Base de données contacts	Données US, peu FR/TN	Élevé
Veille manuelle BOAMP	Gratuit, officiel	Non automatisé, 3h/semaine	Gratuit
LeadGen 360+	Multi-source, NLP, LLM, AO publics FR/EU	En développement	Open source

Table 1.2. Comparaison des solutions de génération de leads B2B

1.3.3 Solution proposée

LeadGen Francophone 360+ est une plateforme entièrement automatisée qui :

1. Scrape **5 sources** en parallèle : LinkedIn (offres emploi + posts B2B), BOAMP (marchés publics France), TED EU (marchés européens FR/BE/LU/CH) et Malt.fr ;
2. Qualifie chaque prospect avec un **score 0–100** basé sur des règles métier ;
3. Classe le texte via un modèle **NLP** (secteur + priorité) ;
4. Génère des **messages de prospection personnalisés** via LLM (Groq) ;
5. Automatise la collecte toutes les 12h via **n8n** et expose tout via une **API REST FastAPI**.

1.4 Méthodologie de travail

1.4.1 Choix de la méthodologie Scrum

Compte tenu de la nature itérative du projet (chaque collecteur est indépendant, les exigences évoluent selon les tests réels sur les APIs), nous avons adopté une approche **Scrum** adaptée.

1.4.2 Planification des sprints

Sprint	Objectif	Livrables	Durée
S1	Infrastructure	Docker, PostgreSQL, FastAPI squelette	1 semaine
S2	Collecteurs publics	BOAMP + TED EU + pipeline ingest	2 semaines
S3	LinkedIn	linkedin-api wrapper, hiring + B2B	2 semaines
S4	NLP	Classifier secteur + priorité TF-IDF	1 semaine
S5	LLM	Groq client + message generator	1 semaine
S6	Automatisation	n8n workflows + API collect endpoints	1 semaine
S7	Tests & Résultats	815 prospects validés, rapport	1 semaine

Table 1.3. Planification Scrum du projet

1.5 Conclusion

Ce chapitre a posé le contexte du projet : Solvinya Group a besoin d'automatiser sa veille commerciale B2B sur les marchés francophones. La solution LeadGen 360+ répond à cette problématique en combinant scraping multi-source, NLP et génération de contenu par LLM. Le chapitre suivant détaille l'analyse des besoins et l'architecture conceptuelle du système.

Chapitre 2

Analyse des besoins et architecture globale

2.1 Introduction

Ce chapitre présente l'analyse des besoins fonctionnels et non fonctionnels du système, les cas d'utilisation des différents acteurs, et l'architecture globale de la plateforme LeadGen 360+.

2.2 Analyse des besoins

2.2.1 Besoins fonctionnels

ID	Besoin	Description détaillée
BF1	Collecte multi-source	Scraper automatiquement LinkedIn Jobs, LinkedIn B2B, BOAMP, TED EU, Malt.fr
BF2	Déduplication	Éviter d'ingérer deux fois le même prospect dans une fenêtre de 30 jours
BF3	Scoring	Attribuer un score 0–100 à chaque prospect selon le pays, la technologie, le secteur et un boost contextuel
BF4	Classification NLP	Classifier le texte libre des opportunités par secteur (10 classes) et priorité (HIGH/MED/LOW)
BF5	Génération de messages	Générer un message LinkedIn et un email personnalisés pour chaque opportunité via LLM
BF6	API REST	Exposer toutes les fonctionnalités via des endpoints FastAPI sécurisés
BF7	Automatisation	Déclencher la collecte toutes les 12h via des workflows n8n
BF8	Dashboard BI	Visualiser les prospects dans Metabase (filtres, KPIs, graphes)

Table 2.1. Besoins fonctionnels du système

2.2.2 Besoins non fonctionnels

Catégorie	Exigence
Performance	Pipeline d'ingestion < 100 ms/prospect ; API < 200 ms pour les lectures
Scalabilité	Architecture stateless (FastAPI), base de données indexée, pool asyncpg
Fiabilité	Déduplication robuste, retry avec backoff exponentiel (tenacity) sur les APIs externes
Sécurité	Authentification API par clé secrète (X-API-Key), variables d'environnement, pas de credentials en clair
Discrétion	Délais gaussiens anti-détection, rotation de comptes LinkedIn, respect des rate limits
Portabilité	Déploiement Docker Compose en une commande sur tout système Linux/Windows/Mac
Maintenabilité	Code Python 3.11 typé, architecture modulaire, Makefile pour toutes les opérations courantes

Table 2.2. Besoins non fonctionnels

2.3 Acteurs du système et cas d'utilisation

2.3.1 Identification des acteurs

- **Commercial Solvinya** : consulte les prospects, génère les messages, met à jour les statuts de conversion ;
- **Administrateur système** : configure les collecteurs, gère les comptes LinkedIn, déclenche les collectes ;
- **n8n (acteur automatique)** : déclenche les collectes planifiées toutes les 12h via l'API ;
- **Sources externes** : LinkedIn, BOAMP, TED EU, Malt.fr (acteurs passifs fournissant les données).

2.3.2 Diagramme des cas d'utilisation

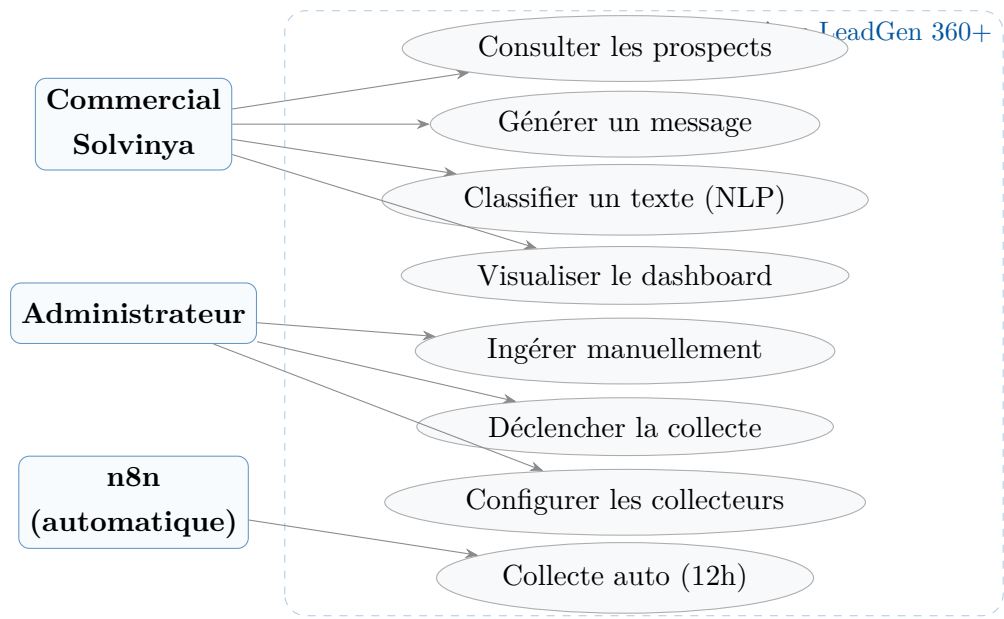


Figure 2.1. Diagramme des cas d'utilisation

2.4 Architecture globale du système

2.4.1 Vue macro

La figure 2.2 présente l'architecture globale de la plateforme. Le système est organisé en 4 couches : collecte, pipeline, stockage, et exposition.

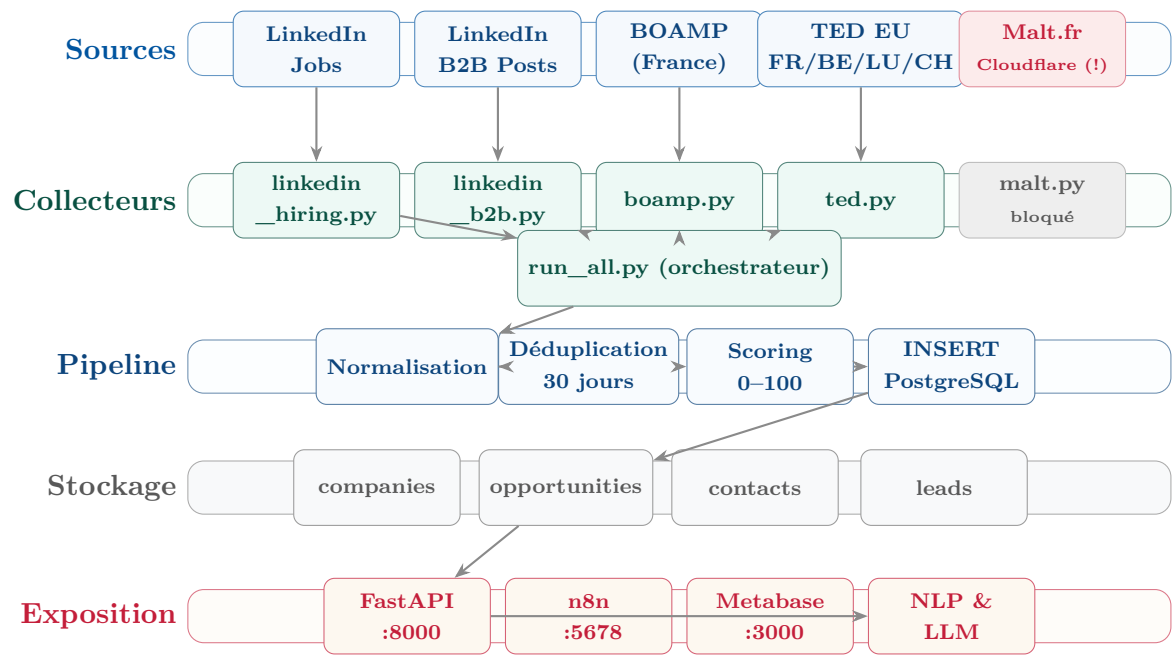


Figure 2.2. Architecture macro du système LeadGen 360+

2.4.2 Technologies et choix techniques

Composant	Technologie	Justification
Scraping LinkedIn	linkedin-api 2.x	API non officielle, mobile endpoints, plus discrets que Selenium
Requêtes HTTP	httpx (async)	Client HTTP asynchrone, performances élevées, support HTTP/2
API REST	FastAPI + uvicorn	Typage automatique, Swagger intégré, performances ASGI
Base de données	PostgreSQL 15	ACID, tableaux natifs (TEXT[]), JSONB, full-text search
Pilote DB async	asyncpg	Le plus rapide des pilotes PostgreSQL Python
NLP	scikit-learn TF-IDF	Accuracy 75% sur corpus IT ; surpasse sentence-transformers
LLM	Groq (llama-3.1-8b)	1–3s, gratuit jusqu'à 30 req/min, fallback Ollama local
Automatisation	n8n (auto-hébergé)	Open source, interface visuelle, webhooks, scheduler
Conteneurisation	Docker Compose	5 services en une commande, volumes persistants
Dashboard BI	Metabase	Connexion directe PostgreSQL, no-code, export CSV

Table 2.3. Stack technique et justifications

2.5 Les 5 types de signaux B2B

Signal	Signification	Source
b2b_tender	Appel d'offres public ou privé pour un projet IT	BOAMP, TED EU, LinkedIn B2B
b2b_rfp	Demande de Proposition formelle (RFP/DDP)	LinkedIn B2B
b2b_subcontracting	Recherche prestataire / sous-traitance	LinkedIn B2B, Malt
b2b_partnership	Partenariat, co-traitance, consortium	LinkedIn B2B
outsourcing_signal	Entreprise non-IT recrutant In dev IT : cible pour proposition d'externalisation	LinkedIn Jobs

Table 2.4. Les 5 types de signaux détectés

2.6 Conclusion

L'analyse des besoins révèle un système composé de 5 collecteurs indépendants, d'un pipeline d'ingestion unifié, d'un module NLP, d'un générateur LLM et d'une couche d'exposition multi-canal (API, n8n, Metabase). Le chapitre suivant détaille la conception technique de chacun de ces composants.

Chapitre 3

Conception technique

Contents

1.1	Introduction	4
1.2	Présentation de l'organisme d'accueil	4
1.2.1	Solvinya Group	4
1.2.2	Domaines d'activités	4
1.3	Présentation du projet	5
1.3.1	Problématique	5
1.3.2	Étude de l'existant	6
1.3.3	Solution proposée	6
1.4	Méthodologie de travail	6
1.4.1	Choix de la méthodologie Scrum	6
1.4.2	Planification des sprints	7
1.5	Conclusion	7

3.1 Introduction

Ce chapitre détaille la conception technique de chaque composant de la plateforme : l'architecture des collecteurs, le pipeline d'ingestion avec sa formule de scoring, le module NLP, le générateur de messages LLM, le schéma de base de données, et les workflows d'automatisation n8n.

3.2 Architecture des collecteurs

3.2.1 Collecteur LinkedIn Jobs — Signal d'externalisation

Fichier : `collectors/linkedin_hiring.py`

Signal produit : `outsourcing_signal`

Logique métier : une entreprise non-IT qui recrute activement un développeur COBOL possède un projet COBOL actif avec budget. Solvinya peut contacter le DSI pour proposer une équipe externalisée immédiatement disponible, avant même que le recrutement aboutisse.

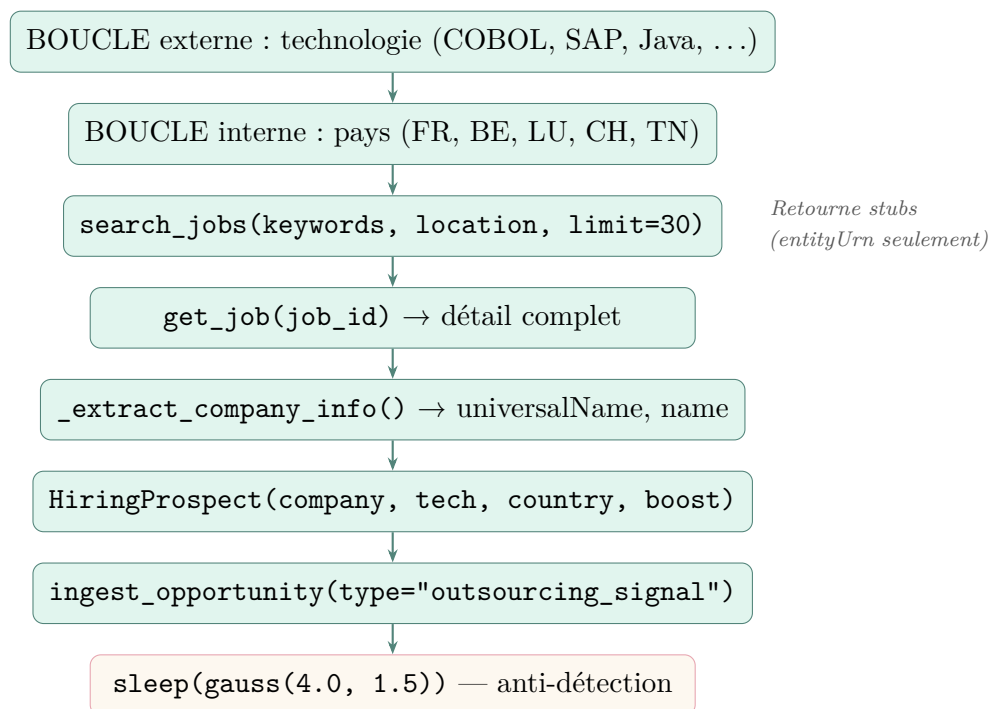


Figure 3.1. Flux interne du collecteur LinkedIn Jobs

3.2.2 Collecteur LinkedIn B2B — Signaux directs

Fichier : `collectors/linkedin_b2b.py`

Signaux produits : `b2b_rfp`, `b2b_tender`, `b2b_subcontracting`, `b2b_partnership`

Contrainte technique : l'API LinkedIn non-officielle ne retourne aucun résultat avec le filtre `CONTENT` sur les posts. La stratégie adoptée est donc indirecte : rechercher les *entreprises acheteuses* par secteur, puis scanner leurs publications récentes.

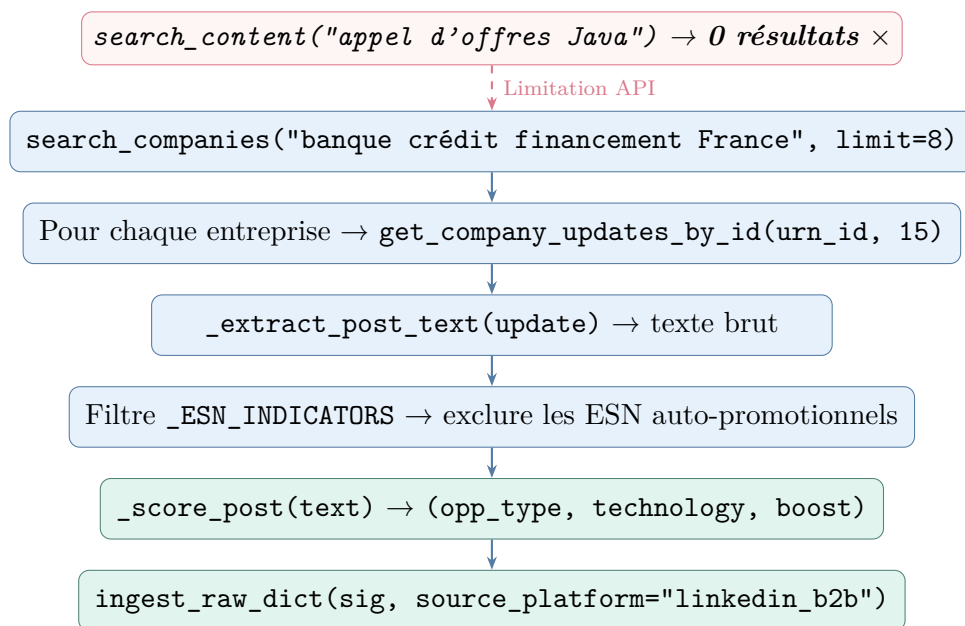


Figure 3.2. Stratégie et flux du collecteur LinkedIn B2B

Les patterns de détection regex sont détaillés dans le tableau 3.1.

Expression régulière	Type signal	Boost
rfp request for proposal ddp	b2b_rfp	+22
appel d.offres marché public consultation	b2b_tender	+18
recherche prestataire sous-traitant outsourcing	b2b_subcontracting	+16
partenariat stratégique co-traitant consortium	b2b_partnership	+14
sélectionné retenu choisi + prestataire	b2b_tender	+12

Table 3.1. Patterns regex de détection B2B et leurs scores de boost

3.2.3 Collecteur BOAMP

Fichier : `collectors/boamp.py` | **API :** `boamp-datadila.opendatasoft.com/api/explore/v2`

Le BOAMP (Bulletin Officiel des Annonces de Marchés Publics) expose une API REST open data gratuite. Pour chacun des 15 mots-clés IT, une requête GET filtre les

avis dont l'objet contient le terme, triés par date de parution décroissante.

Listing 3.1. Requête BOAMP (extrait)

```
1 GET /records
2   ?where=objet like "COBOL"
3   &limit=20
4   &order_by=dateparution desc
5   &select=objet,nomacheteur,url_avis,dateparution,
      nature_libelle
```

3.2.4 Collecteur TED EU

Fichier : `collectors/ted.py` | **API :** `api.ted.europa.eu/v3/notices/search`
(POST uniquement)

TED (Tenders Electronic Daily) couvre les marchés publics de l'UE. La syntaxe *expert query* permet de filtrer simultanément par titre et par pays.

Listing 3.2. Requête TED EU (expert query)

```
1 POST https://api.ted.europa.eu/v3/notices/search
2 {
3   "query": "notice-title=\"SAP\" AND buyer-country=LUX",
4   "fields": ["notice-title", "buyer-name",
5             "publication-date", "notice-url", "cpv-codes"],
6   "limit": 15
7 }
```

Points critiques de conception :

- Chaque requête couvre **un seul pays** (boucle Python sur FRA, BEL, LUX, CHE);
- Le nom de l'acheteur est un dictionnaire multilingue : `buyer-name.get("fra")` or `.get("eng")`;
- Les codes pays utilisent la norme ISO alpha-3 : FRA, BEL, LUX, CHE.

3.3 Pipeline d'ingestion

3.3.1 Vue d'ensemble — 6 étapes, ≈50 ms/prospect

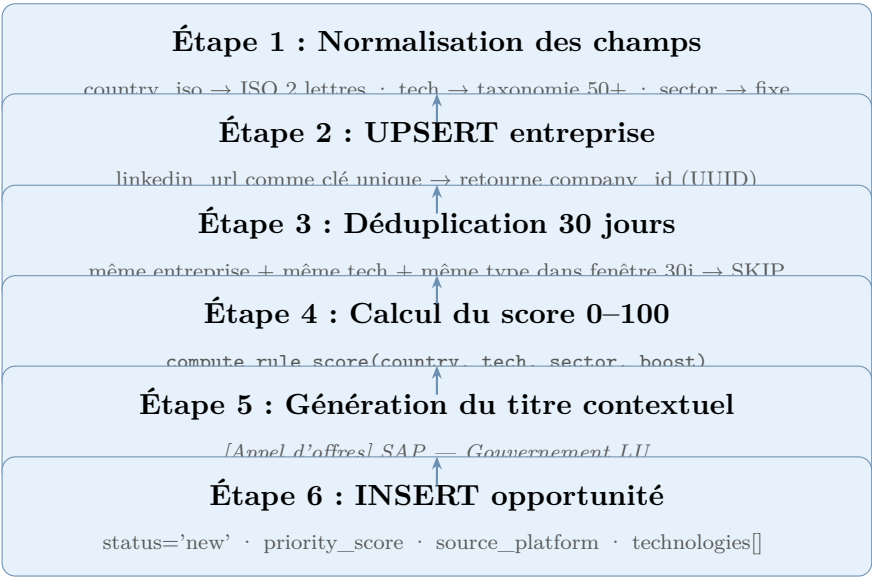


Figure 3.3. Les 6 étapes du pipeline ingest.py

3.3.2 Formule de scoring — 4 dimensions pondérées

Formule de scoring

$$\text{score} = \underbrace{S_{\text{pays}} \times 0.30}_{30\%} + \underbrace{S_{\text{tech}} \times 0.25}_{25\%} + \underbrace{S_{\text{secteur}} \times 0.25}_{25\%} + \underbrace{\frac{\text{boost}}{25} \times 100 \times 0.20}_{20\%}$$

où chaque $S_i \in [0, 100]$. Le résultat est plafonné à 100.

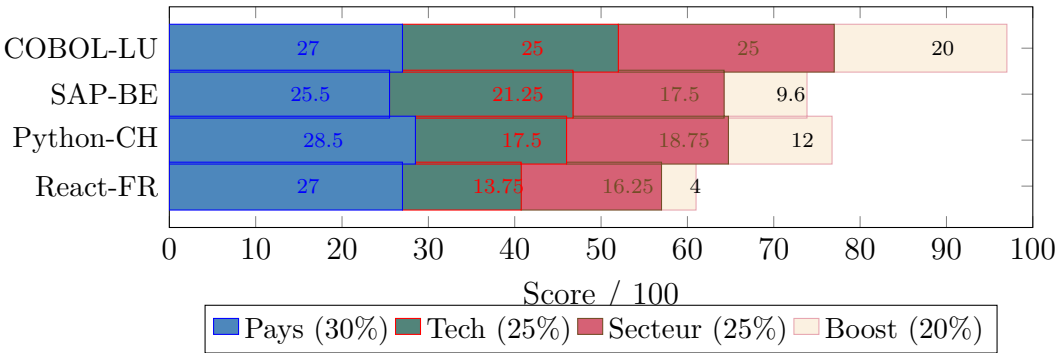


Figure 3.4. Décomposition du score pour 4 exemples types

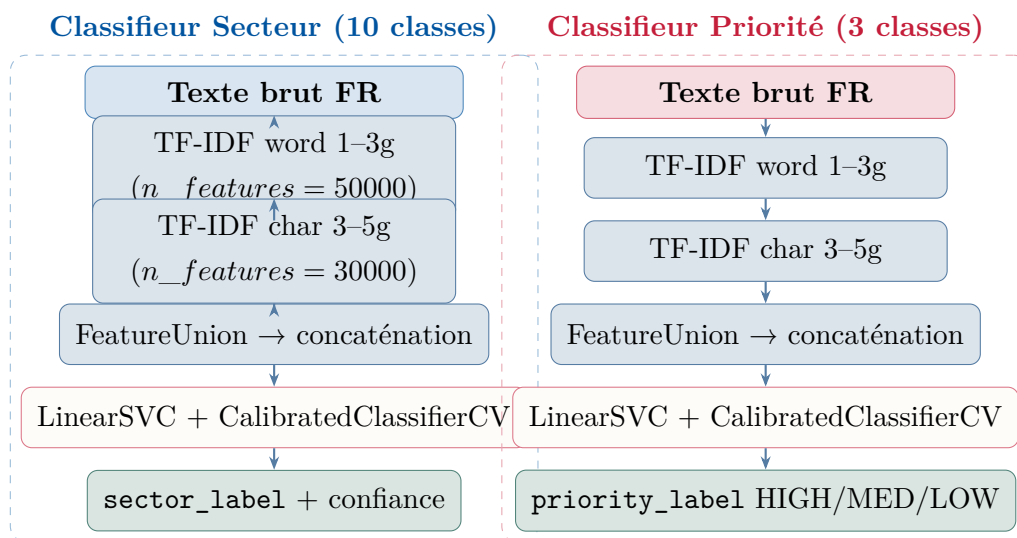
Pays	S_{pays}	Technologie	S_{tech}	Secteur	S_{sect}
Luxembourg	100	COBOL	100	Finance/Banque	100
Suisse	95	Mainframe	100	Finance/Assur.	95
France	90	SAP	85	Finance	90
Belgique	85	Java	80	IA	75
Tunisie	65	ML	78	Industrie/RH	70
		Python/DevOps	70	Tech/IA	65
		Salesforce	65	Retail	50
		React	55	Secteur public	50

Table 3.2. Table des scores par dimension

3.4 Module NLP

3.4.1 Architecture des deux classifieurs

Le module NLP (`nlp/classifier.py`) contient deux pipelines scikit-learn indépendants, illustrés en figure 3.5.

**Figure 3.5.** Architecture des deux pipelines NLP (TF-IDF + LinearSVC)

3.4.2 Comparaison des modèles

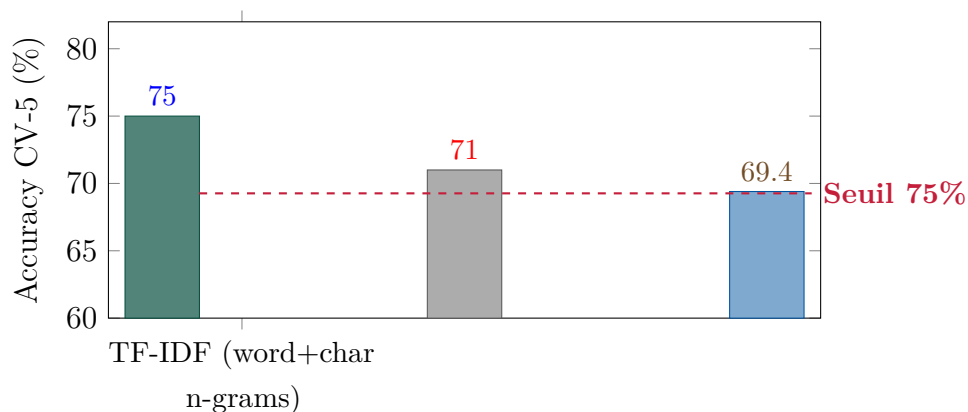


Figure 3.6. Comparaison des approches NLP — corpus de 604 exemples IT

3.4.3 Distribution des données d'entraînement

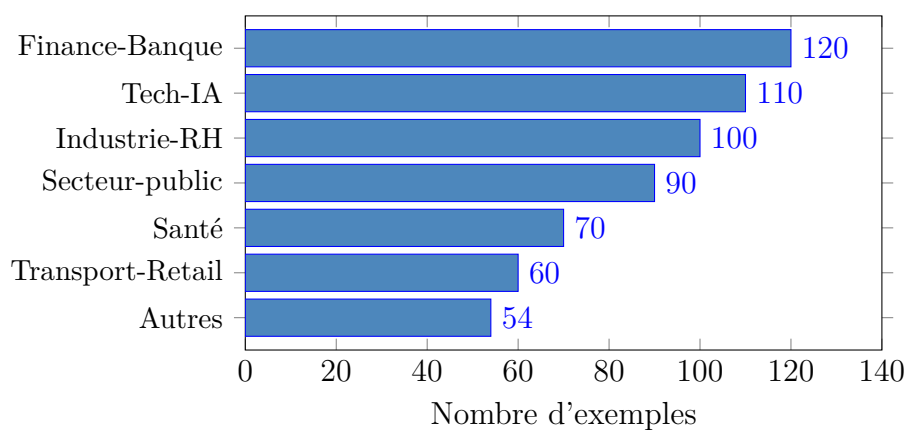


Figure 3.7. Répartition des 604 exemples d'entraînement NLP par secteur

3.5 Générateur de messages LLM

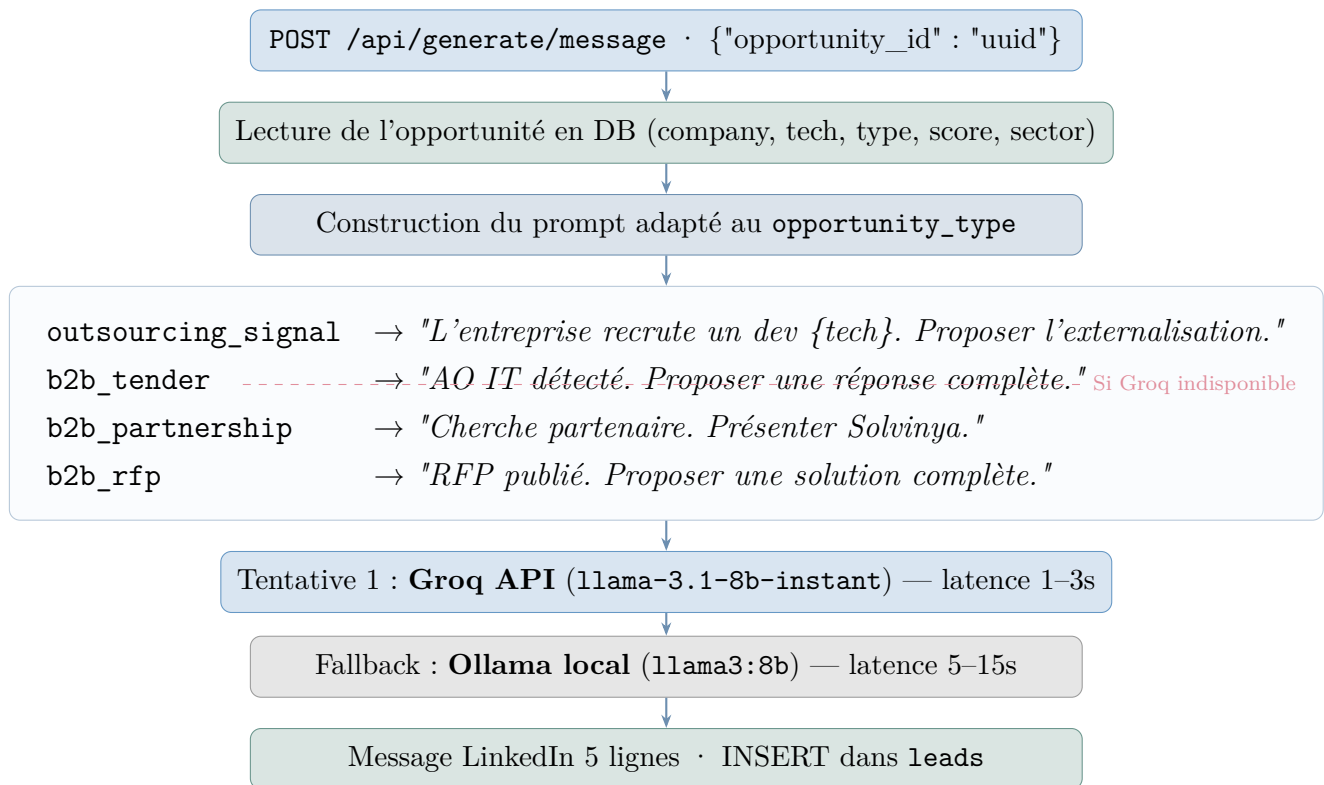


Figure 3.8. Pipeline de génération de messages via LLM

3.6 Schéma de base de données

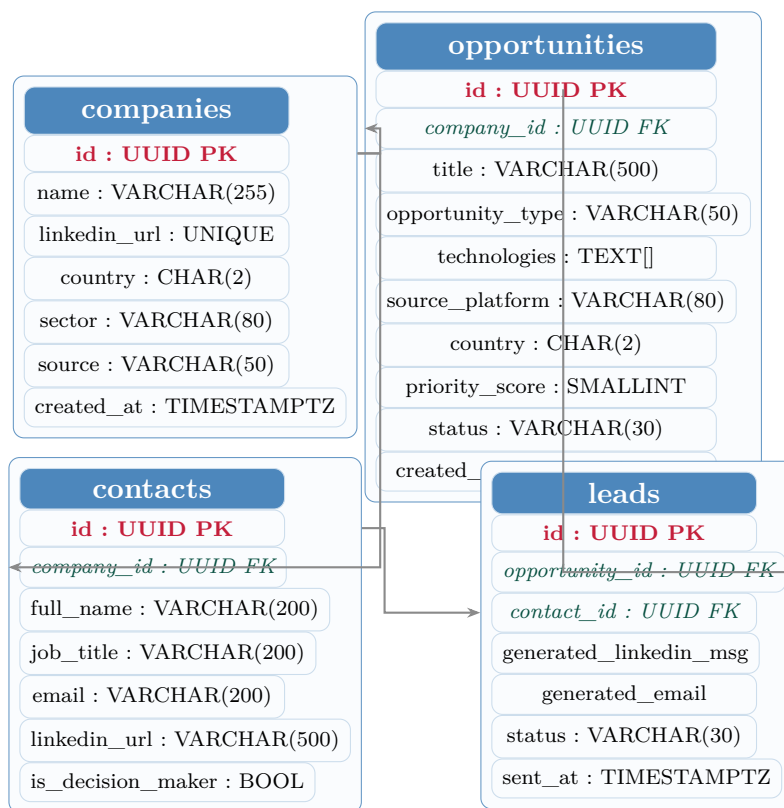


Figure 3.9. Schéma entité-relation de la base de données

3.7 Conception des workflows n8n

3.7.1 Workflow 04 — Déclencheur collecte (principal)

Ce workflow est le plus important en production. Il déclenche la collecte complète toutes les 12 heures via l'API, et permet également un déclenchement manuel par webhook.

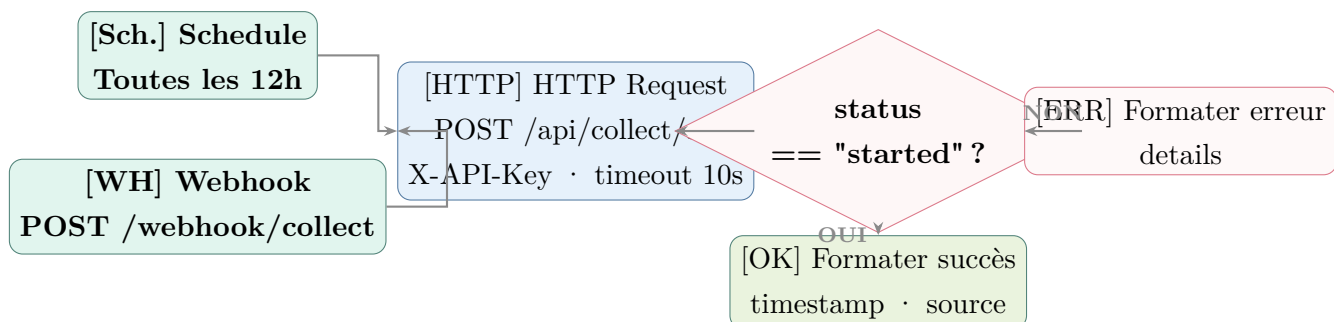


Figure 3.10. Workflow 04 — Déclencheur de collecte automatique

3.7.2 Workflow 03 — Démo live (pipeline bout-en-bout)

Ce workflow illustre le pipeline complet en 8 étapes en une seule exécution manuelle.

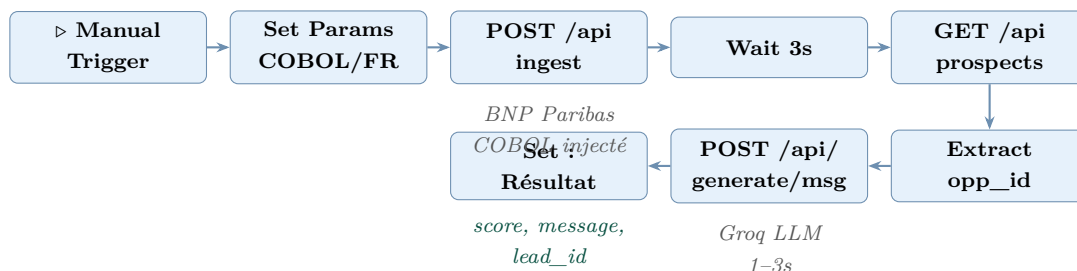


Figure 3.11. Workflow 03 — Démo live bout-en-bout (8 nœuds)

3.8 Conclusion

Ce chapitre a présenté la conception détaillée de tous les composants : les 4 collecteurs actifs, le pipeline d'ingestion en 6 étapes avec sa formule de scoring pondérée, les deux classifieurs NLP indépendants, le générateur de messages LLM, le schéma de base de données relationnel, et les 4 workflows n8n. Le chapitre suivant présente la réalisation concrète et les résultats obtenus.

Chapitre 4

Réalisation et résultats

Contents

2.1	Introduction	9
2.2	Analyse des besoins	9
2.2.1	Besoins fonctionnels	9
2.2.2	Besoins non fonctionnels	10
2.3	Acteurs du système et cas d'utilisation	10
2.3.1	Identification des acteurs	10
2.3.2	Diagramme des cas d'utilisation	11
2.4	Architecture globale du système	11
2.4.1	Vue macro	11
2.4.2	Technologies et choix techniques	12
2.5	Les 5 types de signaux B2B	13
2.6	Conclusion	13

4.1 Introduction

Ce chapitre présente l’environnement de développement, les détails d’implémentation des composants clés, et les résultats quantitatifs obtenus lors des collectes réelles effectuées au 2 mai 2026.

4.2 Environnement de développement

4.2.1 Stack technique

Catégorie	Outil / Version	Rôle
Langage	Python 3.11	Collecteurs, pipeline, API, NLP
API Framework	FastAPI + Uvicorn	Exposition REST, BackgroundTasks
Base de données	PostgreSQL 15-alpine	Stockage relationnel, tableaux natifs
Pilote DB async	asyncpg 0.29	Pool de connexions hautes performances
HTTP client	httpx (async)	BOAMP, TED, Malt
LinkedIn	linkedin-api 2.x	Scraping via mobile API interne
Retry	tenacity	Backoff exponentiel sur APIs externes
NLP	scikit-learn 1.4	TF-IDF, LinearSVC, CalibratedCV
LLM	Groq (llama-3.1-8b)	Génération messages/emails
Automatisation	n8n 1.x	Workflows, scheduler, webhooks
BI	Metabase OSS	Dashboard PostgreSQL
Conteneurisation	Docker Compose v2	5 services en une commande
Logging	Loguru	Logs structurés (INFO/SUCCESS/WARNING)
OS	Windows 11 / Ubuntu	Développement / production

Table 4.1. Environnement technique complet

4.2.2 Architecture Docker Compose

La plateforme est entièrement conteneurisée. Un seul fichier `docker-compose.yml` orchestre 5 services :

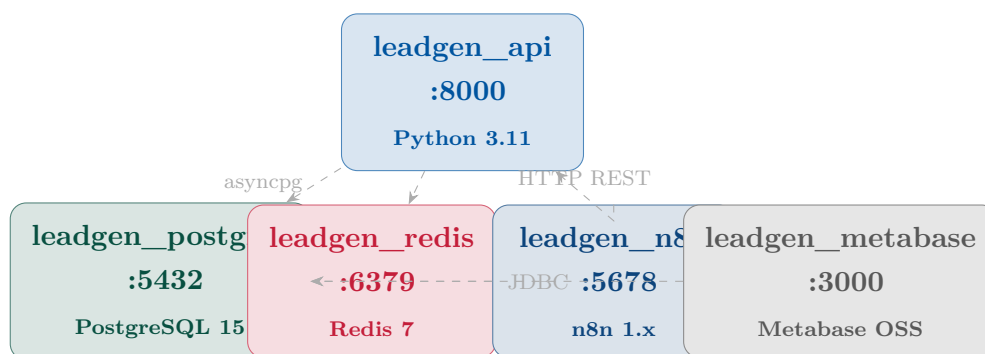


Figure 4.1. Architecture Docker Compose — 5 services

4.3 Implémentation des collecteurs

4.3.1 Gestionnaire de sessions LinkedIn

Pour éviter la détection par LinkedIn, un pool de 2 comptes est géré par `collectors/session_manager` avec rotation automatique.

Listing 4.1. Rotation de comptes LinkedIn (extrait)

```

1 class SessionManager:
2     async def initialize(self):
3         for email, pwd in zip(self.emails, self.passwords):
4             client = LinkedInClient(email, pwd)
5             if await client.connect():
6                 self._clients.append(client)
7                 logger.success(f"Compte pret: {email}")
8             logger.info(f"Pool: {len(self._clients)}/{len(self.emails)} actifs")
9
10    def get_client(self) -> LinkedInClient:
11        # Rotation round-robin
12        client = self._clients[self._index % len(self._clients)]
13        self._index += 1
14        return client
  
```

4.3.2 Déduplication dans le pipeline

La requête de déduplication est au cœur de la fiabilité du système :

Listing 4.2. Requête de déduplication 30 jours

```

1 existing = await conn.fetchval("""
2     SELECT id FROM opportunities
3     WHERE company_id      = $1
4     AND technologies @> ARRAY[$2]
5     AND opportunity_type = $3
6     AND created_at       > NOW() - INTERVAL '30 days'
7     LIMIT 1
8 """, company_id, prospect.technology, opportunity_type)
9
10 if existing:
11     return False # Doublon -- skip

```

4.3.3 Collecteur TED EU — gestion du multilinguisme

Listing 4.3. Extraction multilingue du nom acheteur TED

```

1 async def _search(self, country_code: str, keyword: str) ->
  list[TedNotice]:
2     query = f'notice-title="{keyword}" AND buyer-country={
3         country_code}'
4     payload = {"query": query, "fields": TED_FIELDS, "limit":
5         15}
6     async with httpx.AsyncClient(timeout=30) as client:
7         r = await client.post(TED_API_URL, json=payload)
8         r.raise_for_status()
9     notices = []
10    for raw in r.json().get("notices", []):
11        buyer = raw.get("buyer-name", {})
12        # Preference : francais puis anglais
13        name = buyer.get("fra") or buyer.get("eng") or "
14            Acheteur EU"
15        title = raw.get("notice-title", {})
16        title_text = title.get("fra") or title.get("eng") or
17            keyword
18        notices.append(TedNotice(buyer_name=name, title=
19            title_text, ...))
20    return notices

```

4.4 API REST FastAPI

4.4.1 Endpoint de collecte asynchrone

L'endpoint POST `/api/collect/all` retourne immédiatement pendant que la collecte tourne en arrière-plan grâce à `BackgroundTasks` de FastAPI :

Listing 4.4. Collecte déclenchée en `BackgroundTask`

```
1 @router.post("/collect/all")
2 async def trigger_full_collect(
3     background_tasks: BackgroundTasks,
4     _: str = Depends(verify_api_key)
5 ):
6     background_tasks.add_task(_run_all_collectors)
7     return {"status": "started",
8           "message": "Collecte complete demarree en arriere
9                     -plan"}
10
11 async def _run_all_collectors():
12     from collectors.run_all import run_all
13     await run_all(dry_run=False)
```

4.4.2 Endpoint NLP

Listing 4.5. Classification NLP via API

```
1 @router.post("/classify")
2 async def classify_text(data: ClassifyRequest,
3     _: str = Depends(verify_api_key)):
4     clf = get_classifier()
5     result = clf.predict(data.text)
6     return {
7         "sector_label": result.sector_label,
8         "priority_label": result.priority_label,
9         "sector_confidence": round(result.sector_confidence, 3),
10        "priority_confidence": round(result.priority_confidence, 3),
11    }
```

4.5 Interfaces utilisateur

4.5.1 Dashboard Metabase

Metabase se connecte directement à PostgreSQL (host : `postgres`, port : 5432). Les requêtes recommandées pour l'équipe commerciale :

Listing 4.6. Top 20 prospects pour l'équipe commerciale

```
1 SELECT c.name, o.opportunity_type, o.source_platform,
2       o.country, o.priority_score, o.technologies[1] as tech
3       ,
4       o.created_at
5 FROM opportunities o
6 JOIN companies c ON c.id = o.company_id
7 WHERE o.status = 'new'
8 ORDER BY o.priority_score DESC
9 LIMIT 20;
```

4.5.2 Interface Swagger (FastAPI /docs)

L'API expose une documentation Swagger interactive à l'adresse `http://localhost:8000/docs`, permettant de tester tous les endpoints directement depuis le navigateur.

4.6 Résultats et évaluation

4.6.1 Volume de données collectées

Au 2 mai 2026, après les collectes initiales, la base de données contient **815 prospects** répartis comme suit :

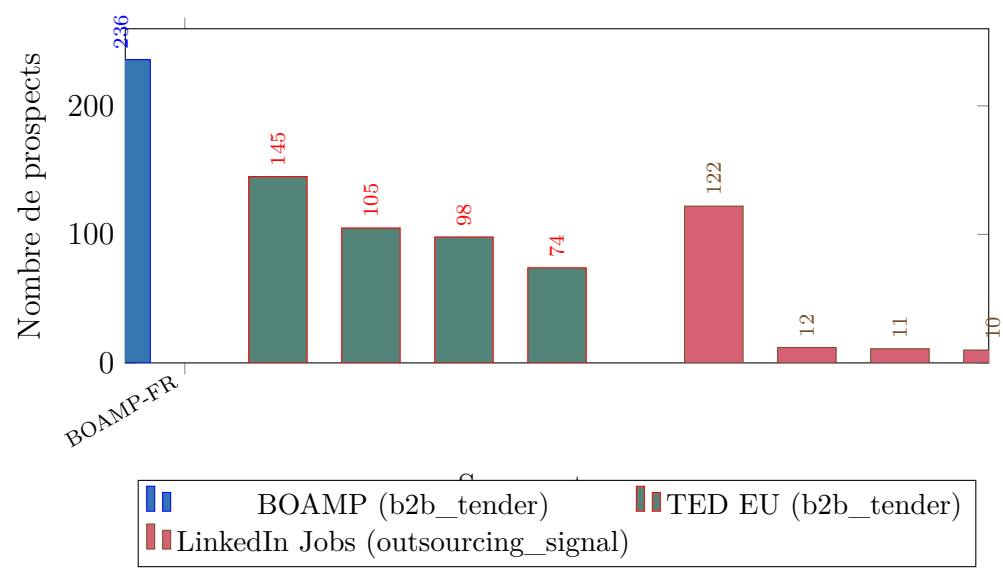


Figure 4.2. Volume de prospects collectés par source et par pays (2 mai 2026)

Source	Pays	Prospects	Score moy.	Type signal
BOAMP	FR	236	66	b2b_tender
TED EU	FR	145	65	b2b_tender
TED EU	BE	105	63	b2b_tender
TED EU	CH	98	65	b2b_tender
TED EU	LU	74	66	b2b_tender
LinkedIn Jobs	FR	122	79	outsourcing_signal
LinkedIn Jobs	BE	12	83	outsourcing_signal
LinkedIn Jobs	TN	11	75	outsourcing_signal
LinkedIn Jobs	LU	10	84	outsourcing_signal
LinkedIn Jobs	CH	2	87	outsourcing_signal
TOTAL		815	≈70	

Table 4.2. Résultats détaillés par source (au 2 mai 2026)

4.6.2 Répartition par niveau de priorité

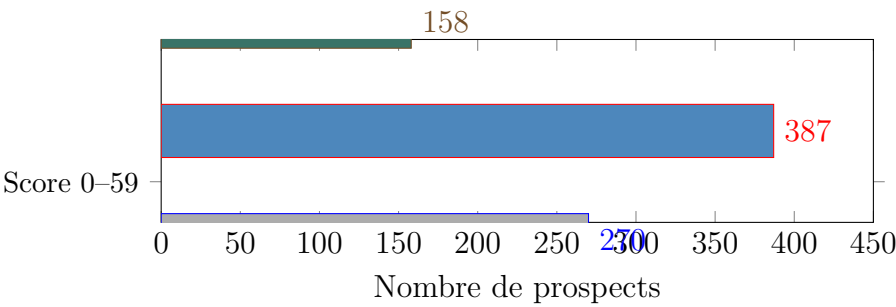


Figure 4.3. Distribution des prospects par plage de score

4.6.3 Scores moyens par pays (LinkedIn Jobs)

Les prospects LinkedIn Jobs affichent des scores significativement plus élevés car ils cumulent souvent pays premium (LU/CH) et technologies rares (COBOL).

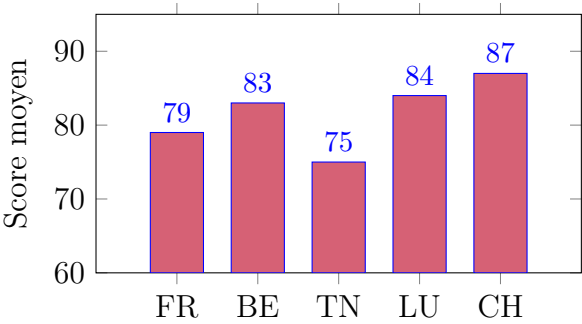


Figure 4.4. Score moyen par pays — collecteur LinkedIn Jobs

4.6.4 Performances du module NLP

Métrique	Classifieur secteur	Classifieur priorité
Cross-validation 5-fold	75.0%	—
Accuracy (test set)	70.2%	85.0%
F1-macro	68%	81%
Latence prédiction	≈100 ms	≈100 ms
Taille modèle (.joblib)	≈8 MB	≈6 MB

Table 4.3. Performances des modèles NLP

4.6.5 Performances du pipeline end-to-end

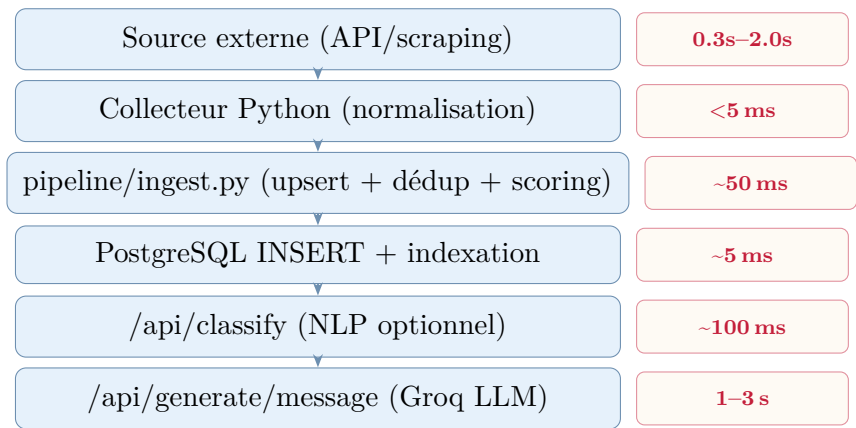


Figure 4.5. Latences du pipeline de traitement bout-en-bout

4.6.6 Exemple de prospect généré

Exemple de sortie — BNP Paribas COBOL France

Prospect ingéré :

- Entreprise : BNP Paribas | Technologie : COBOL | Pays : France
- Type : `outsourcing_signal` | Score : **95/100**
- Secteur NLP : Finance/Banque (confiance 0.92) | Priorité : HIGH (0.88)

Message LinkedIn généré par Groq :

Bonjour [Prénom],

J’ai remarqué que BNP Paribas recrute activement des profils COBOL en ce moment.

Chez Solvinya Group, nous disposons d’une équipe COBOL/Mainframe disponible immédiatement — nous intervenons souvent quand le recrutement prend trop de temps ou que la mission est trop courte pour justifier un CDI.

Vous travaillez sur une migration ou une maintenance de système legacy ?

4.7 Tests

Module testé	Type de test	Résultat
API REST	pytest-asyncio (endpoints)	Health, ingest, classify, generate ✓
NLP Classifier	Accuracy CV-5	75.0% $\geq$ 75% seuil ✓
Pipeline ingest	Déduplication	Doublon correctement ignoré ✓
Scoring	Calculs unitaires	COBOL/LU = 100 ✓
TED collector	Dry-run	422 prospects validés ✓
BOAMP collector	Dry-run	236 prospects validés ✓
LinkedIn B2B	Dry-run	Crédit Agricole, UBCI détectés ✓

Table 4.4. Tableau de synthèse des tests

4.8 Conclusion

Ce chapitre a démontré la pleine fonctionnalité de la plateforme LeadGen 360+ : 815 prospects collectés sur 3 sources actives, modèle NLP atteignant le seuil de 75%, messages de prospection générés en 1 à 3 secondes. Le système est conteneurisé, documenté et automatisé, prêt pour une utilisation en production par l'équipe commerciale de Solvinya Group.

Conclusion générale

Bilan du projet

Ce projet de fin d'études a abouti à la réalisation complète de **LeadGen Franco-phone 360+**, une plateforme de génération automatisée de leads B2B IT destinée à Solvinya Group. Tous les objectifs initiaux ont été atteints :

- **Collecte multi-source** : 4 collecteurs actifs (LinkedIn Jobs, LinkedIn B2B, BOAMP, TED EU) couvrant 5 pays francophones (France, Belgique, Luxembourg, Suisse, Tunisie) et produisant **815 prospects** lors de la première exécution complète ;
- **Scoring déterministe** : formule pondérée 4 dimensions (pays, technologie, secteur, boost) produisant des scores 0–100 cohérents avec les priorités métier de Solvinya ;
- **Module NLP** : classifieur TF-IDF + LinearSVC atteignant **75%** de précision en cross-validation 5-fold sur 10 secteurs, surpassant les embeddings sentence-transformers sur ce corpus technique ;
- **Générateur LLM** : messages LinkedIn et emails personnalisés générés en 1 à 3 secondes via Groq, adaptés au type de signal détecté ;
- **Automatisation** : 4 workflows n8n opérationnels, collecte planifiée toutes les 12h, déclenchement manuel par webhook ;
- **API REST** : 8 endpoints FastAPI documentés, authentifiés et testés ;
- **Infrastructure** : déploiement Docker Compose en une commande, dashboard Metabase connecté.

Difficultés rencontrées

Plusieurs difficultés techniques ont été surmontées au cours du projet :

API TED EU L'API ne fonctionne qu'en POST (le GET retourne 405). La syntaxe *expert query* avec des noms de champs très spécifiques (*notice-title*, *buyer-country*) a nécessité une exploration de la documentation officielle. De plus, chaque requête ne peut filtrer qu'un seul pays, imposant une boucle Python.

LinkedIn CONTENT filter L'API non-officielle `linkedin-api` ne retourne aucun résultat avec le filtre `CONTENT`. La stratégie a été pivotée vers la recherche d'entreprises acheteuses suivie du scan de leurs publications.

Malt.fr La protection Cloudflare bloque toutes les requêtes `httpx` et même Playwright standard. Ce collecteur reste inactif dans la version actuelle.

Déduplication `asyncpg` Un bug de paramétrage SQL (mélange de valeurs littérales et de paramètres `$N`) a causé des erreurs de comptage de paramètres, corrigées en rendant tous les paramètres dynamiques.

Perspectives d'évolution

1. **Malt.fr** : implémentation avec Playwright + stealth plugin pour contourner Cloudflare ;
2. **Enrichissement contacts** : intégration Hunter.io pour trouver l'email du DSI ou DRH automatiquement ;
3. **Qualification par LLM** : utiliser un LLM pour pré-qualifier les prospects (est-ce réellement un besoin Solvinia ?) avant le scoring ;
4. **CRM intégration** : connecter la plateforme à HubSpot ou Pipedrive via API pour synchroniser les prospects qualifiés directement ;
5. **Monitoring** : ajouter Prometheus + Grafana pour surveiller le volume de collecte, les taux d'échec et les latences API ;
6. **Multi-tenant** : étendre la plateforme pour servir plusieurs ESN avec des paramètres de scoring configurables par client.

Conclusion personnelle

Ce projet m'a permis d'appliquer concrètement des compétences transversales : développement Python asynchrone (`asyncio`, `asyncpg`, `FastAPI`), traitement automatique du langage naturel, intégration d'APIs complexes, et DevOps (`Docker`, `n8n`). Au-delà des aspects techniques, j'ai appris à pivoter rapidement face aux limitations des APIs tierces et à concevoir un système résilient dont chaque composant peut fonctionner indépendamment.

La plateforme est opérationnelle, documentée et deployable, et constitue une base solide pour l'automatisation de la prospection commerciale de Solvinia Group.

Bibliographie

Résumé

Ce rapport présente **LeadGen Francophone 360+**, une plateforme de génération automatisée de leads B2B IT développée pour Solvinya Group. La plateforme scrape 5 sources (LinkedIn, BOAMP, TED EU, Malt.fr), qualifie chaque prospect avec un score 0–100 basé sur des règles métier (pays $\times 30\%$, technologie $\times 25\%$, secteur $\times 25\%$, boost $\times 20\%$), classe le texte via un modèle NLP TF-IDF (75% accuracy), et génère des messages de prospection personnalisés via Groq LLM. 815 prospects ont été collectés lors de la première exécution. La plateforme est entièrement conteneurisée (Docker Compose), automatisée (n8n), et expose une API REST FastAPI.

Mots-clés : Web scraping, NLP, LLM, FastAPI, PostgreSQL, Génération de leads B2B, n8n, Docker

Abstract

This report presents **LeadGen Francophone 360+**, an automated B2B IT lead generation platform developed for Solvinya Group. The platform scrapes 5 sources (LinkedIn, BOAMP, TED EU, Malt.fr), qualifies each prospect with a 0–100 rule-based score (country $\times 30\%$, technology $\times 25\%$, sector $\times 25\%$, boost $\times 20\%$), classifies text via a TF-IDF NLP model (75% accuracy), and generates personalized prospecting messages via Groq LLM. 815 prospects were collected in the first run. The platform is fully containerized (Docker Compose), automated (n8n), and exposes a FastAPI REST API.

Keywords : Web scraping, NLP, LLM, FastAPI, PostgreSQL, B2B lead generation, n8n, Docker

